Proving BLEU and chrF Against a Hidden Reference: Candidate-Compiled Automata as STARK Constraints

Abstract

Benchmarks with private test sets let an evaluator report scores that nobody else can recompute. We show how an evaluator can prove the sufficient statistics of BLEU and chrF for a public model output against a reference that stays hidden behind a salted hash commitment. The candidate output is compiled into a deterministic automaton over its own n-grams, a prefix trie whose states record which candidate n-gram the reference has just matched, and saturating unary counters on the trie nodes realise the clip $\min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})$ with low-degree polynomial constraints and no comparison gadget. Together with an in-circuit Rescue-Prime sponge that binds the reference to its commitment, these constraints form an AIR that we prove with Plonky3's zero-knowledge STARK over Goldilocks. On the WMT22 English–German test set (2,037 sentences, two systems), all 4,074 sentence proofs verify and their statistics equal sacreBLEU's; corpus BLEU and chrF recomputed from the proved statistics alone match sacreBLEU exactly. A sentence-level BLEU proof takes 97 ms to generate (median, 16 cores), 11.3 ms to verify and 239 KiB, at an estimated 106 bits of conjectured and 95 bits of proven (list-decoding) soundness. Proving the same statement in the RISC Zero zkVM takes a median of 18.4 s per sentence with a SHA-256 commitment and 223 s with the same Rescue commitment (10 sentences).

1 Introduction

Hidden test sets are a standard defence against benchmark contamination and overfitting [Oren et al., 2024, Singh et al., 2025]: the benchmark owner keeps the references and publishes only scores. The price is verifiability. A reader of a leaderboard cannot check that a reported BLEU score was computed on the published references with the stated tokenization and formula [Post, 2018].

Zero-knowledge proofs remove this trade-off in principle. The owner commits to the references once; for every evaluated output the owner (or any party holding the references) proves that the reported statistics are those of the committed references, and the proof reveals nothing else. Prior work proves model inference or whole evaluation pipelines in zero knowledge [South et al., 2024, Sun et al., 2024, Chen et al., 2024]. Here we focus on the metric step and ask how cheaply the standard string metrics of machine translation can be proved when the reference is secret.

Claim. Compiling the *public* candidate into a weighted automaton over its own n-grams yields an AIR of degree 7 whose width is linear in the candidate length and whose height is the reference length plus one 8-row block, rounded up to a power of two. With an established STARK (Plonky3, hiding FRI) this proves BLEU and chrF sufficient statistics against a committed hidden reference with median proving times of 97 ms (BLEU) and 128 ms (chrF) per sentence and verification in milliseconds, and the statistics agree exactly with sacreBLEU on a full WMT test set.

Contributions.

1. A compilation of clipped n-gram counting into AIR constraints: a candidate-derived prefix-trie automaton evaluated over the reference, plus unary saturating counters that implement clipping without range checks (Section 4), with a soundness argument (Lemma 1).
2. An implementation on Plonky3 0.8 with zero-knowledge FRI and an in-AIR Rp64_256 (Rescue-Prime) sponge whose digest is the reference commitment, plus the same statement as a RISC Zero guest program as a baseline (Section 5).
3. An evaluation on WMT22 en–de: exact agreement with sacreBLEU on 4,074 BLEU proofs and 400 chrF proofs (200 of them zero-knowledge), proving and verification cost, proof size, comparison with the zkVM, and Plonky3's own soundness estimates for the configuration (Section 6).

2 Background and Related Work

BLEU and chrF. BLEU [Papineni et al., 2002] and chrF [Popović, 2015] are functions of per-order counts: for each order n, the number of candidate n-grams, the number of reference n-grams (for chrF), and the clipped matches $m_n = \sum_g \min(c_{\text{cand}}(g), c_{\text{ref}}(g))$ over distinct candidate n-grams g, together with the two lengths. sacreBLEU computes sentence and corpus scores from exactly these statistics with fixed tokenization (13a words for BLEU, characters without whitespace for chrF) [Post, 2018]. Candidate-side counts are public in our setting; the m_n and the reference length are what must be proved.

Weighted automata. Counting occurrences of a fixed pattern is a rational series computed by a weighted automaton over the counting semiring [Schützenberger, 1961, Droste et al., 2009, Mohri, 2009]. A bank of such automata, one per candidate n-gram, determinises into the prefix trie of the candidate's n-grams; this is the structure we encode.

STARKs. A STARK proves that an execution trace satisfies an algebraic intermediate representation (AIR), a set of polynomial constraints between consecutive rows [Ben-Sasson et al., 2019], using the FRI low-degree test [Ben-Sasson et al., 2018]. Concrete soundness is analysed under the ethSTARK conjecture [Ben-Sasson, 2021] or proven via proximity gaps [Ben-Sasson et al., 2020, Haböck, 2022], and the non-interactive version relies on a Fiat–Shamir analysis [Block et al., 2023]. Under-constrained circuits and weak Fiat–Shamir transcripts are recurring causes of real-world SNARK vulnerabilities [Dao et al., 2023, Chaliasos et al., 2024]; we build on an existing proof system, limit our own code to the AIR, and make every AIR constant that depends on the instance part of the public values, which Plonky3 absorbs into the transcript before drawing challenges (Section 4). Plonky3 [Plonky3 contributors, 2023] provides a univariate STARK with a hiding FRI polynomial commitment following Haböck and Kindi [2024]. We commit to references with Rescue-Prime [Szepieniec et al., 2020, Aly et al., 2020] in the Rp64_256 instantiation; Plonky3's Merkle trees and challenger use Poseidon2 [Grassi et al., 2023].

Verifiable machine learning. South et al. [2024] prove the execution of an evaluation pipeline, including inference, with zkSNARKs. zkCNN [Liu et al., 2021], ZKML [Chen et al., 2024], EZKL [Zkonduit Inc., 2022], Kang et al. [2022] and zkLLM [Sun et al., 2024] prove neural-network inference; ZkAudit [Waiwitlikhit et al., 2024] and Lycklama et al. [2024] prove properties of private data and models for audits, and Abbaszadeh et al. [2024] prove training. These systems target inference, training or data properties; where a metric is proved, it is expressed in a general-purpose circuit.

Our statement composes with them: an inference proof can output a commitment to the candidate, which our AIR takes as public input.

Scope relative to private evaluation. South et al.'s main statement binds private model weights and inference to evaluation over public inputs; its private-benchmark discussion also contemplates undisclosed tests. Our statement instead proves only metric statistics for a public candidate and committed reference; we do not prove inference or claim the first private evaluation protocol. P^4Bench [Zhao et al., 2026] proves knowledge of an answer satisfying a public relation while hiding the answer. Our reference-based clipped-count relation is different from that correctness relation. No matched implementation comparison with either system has been run, so these distinctions establish scope rather than a speed advantage.

zkVMs. General-purpose zkVMs such as RISC Zero [Bruestle et al., 2023] prove arbitrary RISC-V programs, so the metric can be written as ordinary Rust. They are the natural baseline for a special-purpose AIR, and benchmarking suites compare proof systems on common tasks [Ernstberger et al., 2024].

3 Threat Model

A *benchmark owner* holds references $r^{(i)}$ and publishes $d^{(i)} = \text{Rp64_256}(s^{(i)} \,\|\, r^{(i)} \,\|\, 0^*)$ for a uniformly random salt $s^{(i)} \in \mathbb{F}^8$, padded to a fixed length. Tokenization and the token-to-field map (BLAKE2b-64 modulo p) are public. A *prover* (the owner, or an evaluator holding the references) receives a candidate $c^{(i)}$, publishes statistics $(m_1, \ldots, m_N, |r^{(i)}|)$ and a proof. A *verifier* knows $c^{(i)}$ and $d^{(i)}$; it compiles the AIR from $c^{(i)}$ itself and recomputes the candidate digest that the proof carries as a public value. The deployment entry point `verify_committed_statement` additionally compares the proof's reference digest with the independently authenticated owner manifest. The lower-level verifier alone only checks a proof-supplied digest; it does not establish that this is the reference fixed by the owner.

Integrity: a malicious prover cannot make the verifier accept statistics other than those of the committed reference, assuming the soundness of the STARK at the configured parameters and collision resistance of Rp64_256. The prover may choose the candidate, but only before the transcript's challenges exist: the candidate's digest is a public value, so it is fixed when the constraint-combination and out-of-domain challenges are drawn. *Confidentiality:* the proof reveals nothing beyond the public statistics, assuming Plonky3's hiding FRI is zero-knowledge. The statistics themselves reveal how much of the candidate occurs in the reference; publishing scores at all reveals this much.

Out of scope are the provenance of the candidate (it must come from the evaluated model, which needs an inference proof or a trusted evaluation environment), correct tokenization of the reference at commitment time, and whether BLEU or chrF measure what one wants.

4 Approach

4.1 Statement

Fix the maximum order N ($N = 4$ for BLEU over words, $N = 6$ for chrF over characters) and a trace height $T = 8(B + 1)$, a power of two. Token ids are nonzero elements of the Goldilocks field $\mathbb{F}$, $p = 2^{64} - 2^{32} + 1$; 0 is padding. Given a public candidate c and public values $(d, m_1, \ldots, m_N, \ell, h)$

with $h = \mathrm{Rp64_256}(N, T, |c|, c_1, \ldots, c_{|c|})$, the prover shows knowledge of $s \in \mathbb{F}^8$ and $r \in (\mathbb{F} \setminus \{0\})^L$, $L \le 8B$, such that $\mathrm{Rp64_256}(s \,\|\, r \,\|\, 0^{8B-L}) = d$, $\ell = L$, and $m_n = \sum_{g \in G_n} \min(c_c(g), c_r(g))$ for each $n \le N$, where G_n is the set of distinct n-grams of c.

4.2 The candidate automaton

Let $\mathcal{T}$ be the set of distinct k-grams of c for $1 \le k \le N$. Every prefix of a member of $\mathcal{T}$ is in $\mathcal{T}$, so $\mathcal{T}$ is a trie. For $u \in \mathcal{T}$ of depth k let $\mathrm{par}(u)$ be its prefix of length $k-1$ and $\mathrm{last}(u)$ its final token. Reading the reference left to right, the indicator $A_u(t) = [r_{t-k+1..t} = u]$ satisfies

$$A_u(t) = [r_t = u] \ (k = 1), \qquad A_u(t) = A_{\mathrm{par}(u)}(t-1) \cdot A_{\mathrm{last}(u)}(t) \ (k \ge 2).$$

This is the run of a deterministic automaton with one-hot state; each trie node is one boolean trace column, and each step is a product of two columns.

Clipping by unary counters. For a node u with candidate count c_u, the trace holds flags $F_{u,1}, \ldots, F_{u,c_u}$, all zero initially, updated by

$$F_{u,j}(t) = F_{u,j}(t-1) + A_u(t)\, F_{u,j-1}(t-1)\,(1 - F_{u,j}(t-1)), \qquad F_{u,0} \equiv 1.$$

On a match exactly the lowest unset flag turns on, so after reading the reference $\sum_j F_{u,j} = \min(c_u, c_r(u))$. The clip, which would otherwise need a comparison gadget with a bit decomposition, costs c_u columns and one degree-4 constraint per flag. Since $\sum_u c_u$ equals the number of candidate n-gram occurrences, the counter columns total at most $N|c|$.

Equality with constants. For each candidate token v, the indicator $A_v(t) = [r_t = v]$ uses an inverse witness I_v: $A_v(r_t - v) = 0$ and $(r_t - v)I_v = 1 - A_v$. A nonzero indicator $Z(t) = [r_t \ne 0]$ and a running length use the same device.

4.3 Binding the reference to the commitment

The trace runs the Rp64_256 sponge (state width 12, rate 8, 7 rounds [Szepieniec et al., 2020]) in 8-row cycles. Cycle b holds reference block $b-1$ in eight buffer columns (cycle 0 holds the salt in the initial rate). Rows 0..6 of a cycle apply one round each; the transition out of row 7 adds the next block to the rate. A round $H' = M\,(MH^7 + a_1)^{1/7} + a_2$ is enforced with an intermediate column vector Y through $Y^7 = MH^7 + a_1$ (degree 7, every row) and $H' = MY + a_2$ (degree 1, on round rows); round constants and row-type selectors are periodic columns. The capacity is initialised to $(T, 0, 0, 0)$ exactly as winter-crypto's `hash_elements` does, and the digest is the rate prefix on the last row. The token at row t is $\sum_k \mathrm{sel}_k(t)\,\mathrm{BUF}_k(t)$, so the automaton reads exactly the hashed tokens. An activity column, zero during the salt cycle, gates the automaton and counters so that salt elements are never read as tokens.

Binding the candidate. The candidate's token ids and counts appear in the AIR only as constants, and Plonky3's transcript binds the AIR's shape and its public values but not its constraint polynomials. The last four public values are therefore h, the Rp64_256 digest of $(N, T, |c|, c)$, and a first-row constraint pins them to the digest of the candidate the AIR was compiled from. The verifier entry point (`verify_statement`) rebuilds the AIR from the candidate it is given, checks h, and checks that the proof degree is $\log_2 T$ in plain mode or $\log_2 T + 1$ in hiding mode before running the Plonky3 verifier. Because public values are observed after the trace commitment and before the combination challenge α and the out-of-domain point ζ, a prover cannot select candidate ids as a function of ζ without changing h, and hence ζ.

Padding. A transition constraint $\mathrm{ACT} \cdot Z(t{+}1) \cdot (1 - Z(t)) = 0$ forces a zero token inside the reference cycles to be followed by a zero, so the nonzero tokens form a prefix and ℓ counts exactly that prefix.

Lemma 1 (Soundness of the AIR)**.** *If a trace of exactly T rows satisfies all constraints of the AIR compiled from c with public values (d, m, ℓ, h), then h is the digest of $(N, T, |c|, c)$ and there are s and $r \in (\mathbb{F} \setminus \{0\})^{\ell}$ with* Rp64_256$(s \| r \| 0^{8B-\ell}) = d$ *and $m_n = \sum_{g \in G_n} \min(c_c(g), c_r(g))$.*

Proof sketch. Since $x \mapsto x^7$ is a bijection of $\mathbb{F}$ ($\gcd(7, p-1) = 1$), the round constraints determine each row's state from the previous one, so the last-row rate equals the sponge output on the absorbed blocks; take s to be the initial rate and r the buffer contents of cycles $1..B$. The buffer constancy and selection constraints make the token column equal to r in order. By the inverse-witness constraints, $A_v(t) = [r_t = v]$ on every row. The activity column is 0 on rows 0..7 and 1 afterwards, so by induction on t the trie columns satisfy $A_u(t) = [r_{t-k+1..t} = u]$ with all positions in the reference. The counters start at zero, stay boolean, and by induction $F_{u,j}(t) = [\#\{\tau \le t : A_u(\tau) = 1\} \ge j]$ for $j \le c_u$, so $\sum_j F_{u,j}(T-1) = \min(c_u, c_r(u))$; the last-row constraints sum these by order. The nonzero indicator is exact by its inverse witness; the padding constraint makes the nonzero tokens a prefix of the $8B$ absorbed elements, and the length column counts them, so r is that prefix and the remaining absorbed elements are zero. The digest constraint gives h directly. □

The lemma is exact (no soundness error): the STARK contributes the error estimated in Section 6. The two inverse-witness columns are free exactly when their denominator is zero, which does not affect any indicator.

5 Implementation

Plonky3 backend. The AIR is implemented in Rust against Plonky3 0.8.0 (`p3-uni-stark`) in `crates/spectacles-p3`. The configuration uses Goldilocks with its degree-2 extension as challenge field, Poseidon2 (width 8) Merkle trees and duplex challenger, `HidingFriPcs` with salted Merkle leaves and four random codewords, blowup 8, 60 FRI queries, and 16 bits of query grinding. The maximum constraint degree is 7. The Rescue round function is implemented with Plonky3 field types and the constants of winter-crypto 0.13.1's `Rp64_256`; a unit test checks the sponge against winter-crypto, so a reference commitment can be recomputed with an off-the-shelf library.

Owner commitment substitution. A separate application-level experiment fixes a candidate $[11, 22, 11, 33]$, order two and $T = 64$. The owner reference $[11, 22, 44, 55]$ has counts $(2, 1)$; a substituted reference equal to the candidate has counts $(4, 3)$ (independently counted with Python window counters). In both plain and hiding modes, valid proofs for either reference pass internal verification. Only the owner proof passes the verifier supplied with the pre-fixed owner digest; transplanting that digest into the substitute proof is rejected. The fresh offline source build passes all 18 standalone backend tests, including this new test. Small FRI settings are used here to test the application boundary, not the paper's security configuration. This is evidence that external commitment selection matters, not a STARK forgery or a complete soundness audit. Manifest authentication, item coverage and candidate provenance remain outside this API.

Tests. Besides end-to-end prove/verify tests, every proof in the evaluation is re-verified against public values with $m_1 + 1$ and with a modified digest element, and with a different candidate given to the verifier; all three must be rejected. Regression tests reproduce two attacks from an independent

review: candidate ids solved after seeing ζ (a forged proof and the adaptive forging procedure, run with an instrumented copy of the Plonky3 verifier), and a reference with an interior zero; both are rejected. A randomized test adds one to 400 random trace cells outside the salt cycle and the inverse witnesses and checks that each corruption violates a constraint.

zkVM baseline. `zkvm/risc0` contains a RISC Zero 3.0.6 guest that reads the candidate, salt and reference, computes the commitment (Rp64_256 via winter-crypto, or SHA-256 via RISC Zero's accelerated `sha2`), computes the clipped counts with hash maps, and commits the digest and the statistics, together with the candidate, to the journal. Since the journal is bound into the receipt, the zkVM proves the same candidate-bound statement as the AIR. The host uses the default CPU prover; receipts are composite unless stated otherwise.

Supporting artifacts. The repository also contains the SPECTACLES metric library (reference, automaton and arithmetic implementations of the metrics, cross-checked by differential tests) and a Lean 4 development of its semiring and automaton layer and of a small metric-specification DSL (typing, determinism, type preservation), which builds with no `sorry` and one axiom stating STARK soundness for an abstract verifier.

6 Evaluation

We ask: **RQ1** Do the proved statistics equal those of a standard implementation on real data? **RQ2** What do proofs cost? **RQ3** How does the special-purpose AIR compare with a zkVM proving the same statement? **RQ4** What soundness does the configuration provide?

Setup. Data: the WMT22 general test set, English→German [Kocmi et al., 2022], reference A, and the outputs of JDExploreAcademy and Online-B from the WMT22 system-output release (2,037 sentences each; file hashes in `data/wmt22/SHA256SUMS`). Tokenization and expected statistics come from sacreBLEU 2.4.3 with default settings (signatures in `results/inputs/prep_summary.json`). BLEU uses all sentences of both systems; chrF and the zkVM comparison use a fixed random sample of 200 sentences (seed 2026) of JDExploreAcademy. All timings are on Modal CPU containers with 16 physical cores and 32 GiB of memory, for both provers; the Plonky3 prover uses 16 threads. Zero-knowledge proofs use at least 128 trace rows, the smallest height at which Plonky3's hiding budget admits 60 queries. Salts in the evaluation are derived deterministically from public item identifiers so that digests are reproducible; these evaluation commitments are therefore not hiding, whereas a deployment draws salts uniformly at random.

RQ1: agreement. All 4,074 BLEU proofs and all 400 chrF proofs (200 zero-knowledge and 200 non-hiding) verify, and in every case the proved statistics equal sacreBLEU's per-sentence statistics. Recomputing corpus scores from the proved statistics and the public candidate counts gives BLEU 37.79 (JDExploreAcademy) and 38.37 (Online-B), and chrF 64.17 on the sample, identical to sacreBLEU's corpus scores to all printed digits. All tampered public-value vectors and all candidate substitutions are rejected.

RQ2: cost. Table 1 reports prover time (excluding trace generation, which takes under 20 ms), verifier time and serialized proof size. BLEU traces have 128 rows and a median of 175 columns; the longest sentence takes 156 ms. chrF traces are wider because character 6-grams produce larger tries, which raises proof size. Zero knowledge costs 2.2× (BLEU) and 1.8× (chrF) the median non-hiding

Table 1: Plonky3 proofs on WMT22 en–de (median / 90th percentile).

Statement	proofs	prove (ms)	verify (ms)	size (KiB)	width
BLEU, zk, both systems	4,074	97 / 109	11.3 / 14.3	239 / 317	175
BLEU, non-hiding, sample	200	44 / 54	5.9 / 8.6	117 / 195	176
chrF, zk, sample	200	128 / 264	24.2 / 43.3	620 / 1,148	938
chrF, non-hiding, sample	200	71 / 130	19.7 / 36.6	507 / 1,030	938

Table 2: RISC Zero 3.0.6 on the same BLEU statements (medians; ratio = per-sentence RISC Zero / Plonky3-zk prove time).

Prover	n	prove (ms)	verify (ms)	KiB	cycles	ratio
Plonky3 AIR, zk (same sentences)	100	97	11.4	247	–	1
RISC Zero, SHA-256 commitment, composite	100	18,425	17.8	262	524,288	172×
RISC Zero, SHA-256 commitment, succinct	10	13,411	15.1	218	262,144	142×
RISC Zero, Rp64_256 commitment, composite	10	223,068	114.3	1,649	6,291,456	2,312×

proving time: the hiding PCS adds random codewords and doubles the committed domain, and the hiding budget forces at least 128 rows where the non-hiding runs use 64.

RQ3: zkVM baseline. We ran the RISC Zero guest on the first sentences of the fixed sample, padded to the same 128-row reference capacity. All 120 receipts verify and their journals equal sacreBLEU's statistics and the Plonky3 public values; with the Rp64_256 commitment the guest's digest equals the AIR's digest on all 10 items. Table 2 pairs each receipt with the Plonky3 zero-knowledge proof of the same sentence. The zkVM is 172 times slower to prove with SHA-256 and 2,312 times slower with the Rp64_256 commitment on 10 sentences (medians of per-sentence ratios). With the Rp64_256 commitment the guest executes 24.0 times the cycles of the SHA-256 variant on the same sentences (median): SHA-256 runs on RISC Zero's accelerator, while Rescue's inverse S-box is a 64-bit exponentiation emulated on a 32-bit RISC-V core. Median receipt sizes are 262 KiB (composite) and 218 KiB (succinct) against 247 KiB for Plonky3 on the same sentences; verification takes 17.8 ms against 11.4 ms. The zkVM needs no circuit design, so it is the better choice for metrics without a compact automaton formulation.

RQ4: soundness. Plonky3's estimator (`p3_uni_stark::security`) evaluated on the AIR of every BLEU item gives at least 106 bits under the ethSTARK conjecture and at least 95 bits proven in the list-decoding regime (65 bits in the unique-decoding regime); for the chrF sample the minima are 102 and 92 bits. The parameter sweep in `results/p3/security_sweep_*.json` shows that the conjectured level stays at or below 108 bits (zero-knowledge mode) for every query count and blowup we tried, because the estimator's field-size terms for the 128-bit challenge field dominate; additional queries raise only the proven bounds.

7 Limitations

The evaluation covers one language pair and two systems; trace width grows with candidate length, and long candidates with chrF produce proofs above 1 MiB. Reference tokenization happens before commitment and is trusted, and token ids are 64-bit hashes, so two distinct tokens could

in principle share an id (none do among the 11,970 distinct tokens here). The statement covers n-gram metrics; ROUGE-L and other alignment-based metrics need a different AIR. Our AIR and Rescue implementation are not independently audited, and the security levels are those of Plonky3's estimator for the stated configuration. Per-sentence statistics leak overlap information about the reference; aggregating before publication needs a batched statement.

8 Conclusion

A public candidate can be compiled into an automaton over its own n-grams whose run over a hidden, committed reference is a small degree-7 AIR. With Plonky3's zero-knowledge STARK this gives sentence-level BLEU and chrF statistics that match sacreBLEU exactly on a full WMT test set, with median proving times of 97 ms (BLEU) and 128 ms (chrF) and verification in 11.3–24.2 ms per sentence, which is a median of 172 times faster than RISC Zero with a SHA-256 commitment and 2,312 times faster with the same Rescue commitment (10 sentences).

References

Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, pages 4316–4330. ACM, 2024. doi: 10.1145/3658644.3670316.

Abdelrahaman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec. Design of symmetric-key primitives for advanced cryptographic protocols. *IACR Transactions on Symmetric Cryptology*, 2020(3):1–45, 2020. doi: 10.46586/tosc.v2020.i3.1-45.

Eli Ben-Sasson. ethSTARK documentation. Cryptology ePrint Archive, Paper 2021/582, 2021. URL https://eprint.iacr.org/2021/582. Documentation accompanying StarkWare's ethSTARK codebase.

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.14.

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, Lecture Notes in Computer Science, pages 701–732. Springer, 2019. doi: 10.1007/978-3-030-26954-8_23. Full version: "Scalable, transparent, and post-quantum secure computational integrity", IACR ePrint 2018/046.

Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed-Solomon codes. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 900–909. IEEE, 2020. doi: 10.1109/FOCS46700.2020.00088. Journal version: J. ACM 70(5):1–57, 2023, doi:10.1145/3614423.

Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat-Shamir security of FRI and related SNARKs. In Jian Guo and Ron Steinfeld,

editors, *Advances in Cryptology – ASIACRYPT 2023*, Lecture Notes in Computer Science, pages 3–40. Springer, 2023. doi: 10.1007/978-981-99-8724-5_1. IACR ePrint 2023/1071.

Jeremy Bruestle, Paul Gafni, and the RISC Zero Team. RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. https://dev.risczero.com/proof-system-in-detail.pdf, August 2023. Whitepaper (draft dated August 11, 2023). Implementation: https://github.com/risc0/risc0; we use risc0-zkvm 3.0.6 (released July 2026).

Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. SoK: What don't we know? understanding security vulnerabilities in SNARKs. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3855–3872. USENIX Association, 2024. ISBN 978-1-939133-44-1. URL https://www.usenix.org/conference/usenixsecurity24/presentation/chaliasos. arXiv:2402.15293.

Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. ZKML: An optimizing system for ML inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, pages 560–574. ACM, 2024. doi: 10.1145/3627703.3650088.

Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. Weak Fiat-Shamir attacks on modern proof systems. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 199–216. IEEE, 2023. doi: 10.1109/SP46215.2023.10179408. IACR ePrint 2023/691.

Manfred Droste, Werner Kuich, and Heiko Vogler, editors. *Handbook of Weighted Automata*. Monographs in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2009. doi: 10.1007/978-3-642-01492-5.

Jens Ernstberger, Stefanos Chaliasos, George Kadianakis, Sebastian Steinhorst, Philipp Jovanovic, Arthur Gervais, Benjamin Livshits, and Michele Orrù. zk-Bench: A toolset for comparative evaluation and performance benchmarking of SNARKs. In Clemente Galdi and Duong Hieu Phan, editors, *Security and Cryptography for Networks (SCN 2024)*, Lecture Notes in Computer Science, pages 46–72. Springer, 2024. doi: 10.1007/978-3-031-71070-4_3. IACR ePrint 2023/1503.

Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology – AFRICACRYPT 2023*, Lecture Notes in Computer Science, pages 177–203. Springer, 2023. doi: 10.1007/978-3-031-37679-5_8. IACR ePrint 2023/323.

Ulrich Haböck. A summary on the FRI low degree test. Cryptology ePrint Archive, Paper 2022/1216, 2022. URL https://eprint.iacr.org/2022/1216.

Ulrich Haböck and Al Kindi. A note on adding zero-knowledge to STARKs. Cryptology ePrint Archive, Paper 2024/1037, 2024. URL https://eprint.iacr.org/2024/1037.

Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. Scaling up trustless DNN inference with zero-knowledge proofs. arXiv preprint arXiv:2210.08674, 2022. URL https://arxiv.org/abs/2210.08674.

Tom Kocmi, Rachel Bawden, Ondřej Bojar, Anton Dvorkovich, Christian Federmann, Mark Fishel, Thamme Gowda, Yvette Graham, Roman Grundkiewicz, Barry Haddow, Rebecca Knowles, Philipp Koehn, Christof Monz, Makoto Morishita, Masaaki Nagata, Toshiaki Nakazawa, Michal Novák, Martin Popel, and Maja Popović. Findings of the 2022 conference on machine translation (WMT22). In *Proceedings of the Seventh Conference on Machine Translation (WMT)*, pages 1–45,

Abu Dhabi, United Arab Emirates (Hybrid), 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.wmt-1.1. URL https://aclanthology.org/2022.wmt-1.1/.

Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*, pages 2968–2985. ACM, 2021. doi: 10.1145/3460120.3485379.

Hidde Lycklama, Alexander Viand, Nicolas Küchler, Christian Knabenhans, and Anwar Hithnawi. Holding secrets accountable: Auditing privacy-preserving machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024. URL https://www.usenix.org/conference/usenixsecurity24/presentation/lycklama. arXiv:2402.15780.

Mehryar Mohri. Weighted automata algorithms. In Manfred Droste, Werner Kuich, and Heiko Vogler, editors, *Handbook of Weighted Automata*, Monographs in Theoretical Computer Science. An EATCS Series, pages 213–254. Springer, Berlin, Heidelberg, 2009. doi: 10.1007/978-3-642-01492-5_6.

Yonatan Oren, Nicole Meister, Niladri S. Chatterji, Faisal Ladhak, and Tatsunori Hashimoto. Proving test set contamination in black-box language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=KS8mIvetg2. Oral. arXiv:2310.17623.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL https://aclanthology.org/P02-1040/.

Plonky3 contributors. Plonky3: A toolkit for polynomial IOPs. https://github.com/Plonky3/Plonky3, 2023. Software, Apache-2.0. Repository created March 2023; we use crates version 0.8.0 (released September 2026).

Maja Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal, 2015. Association for Computational Linguistics. doi: 10.18653/v1/W15-3049. URL https://aclanthology.org/W15-3049/.

Matt Post. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 186–191, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-6319. URL https://aclanthology.org/W18-6319/.

Marcel-Paul Schützenberger. On the definition of a family of automata. *Information and Control*, 4 (2–3):245–270, 1961. doi: 10.1016/S0019-9958(61)80020-X.

Shivalika Singh, Yiyang Nan, Alex Wang, Daniel D'Souza, Sayash Kapoor, Ahmet Üstün, Sanmi Koyejo, Yuntian Deng, Shayne Longpre, Noah A. Smith, Beyza Ermis, Marzieh Fadaee, and Sara Hooker. The leaderboard illusion. In *Advances in Neural Information Processing Systems (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. URL https://openreview.net/forum?id=4Ae8edNqm0. arXiv:2504.20879.

Tobin South, Alexander Camuto, Shrey Jain, Shayla Nguyen, Robert Mahari, Christian Paquin, Jason Morton, and Alex 'Sandy' Pentland. Verifiable evaluations of machine learning models using zkSNARKs. arXiv preprint arXiv:2402.02675, 2024. URL https://arxiv.org/abs/2402.02675.

Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, pages 4405–4419. ACM, 2024. doi: 10.1145/3658644.3670334. arXiv:2404.16109.

Alan Szepieniec, Tomer Ashur, and Siemen Dhooghe. Rescue-Prime: a standard specification (SoK). Cryptology ePrint Archive, Paper 2020/1143, 2020. URL https://eprint.iacr.org/2020/1143.

Suppakit Waiwitlikhit, Ion Stoica, Yi Sun, Tatsunori Hashimoto, and Daniel Kang. Trustless audits without revealing data or models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 49808–49821. PMLR, 2024. arXiv:2404.04500.

Enhan Zhao, Yuanrui Zhang, Wei Wu, and Di He. P4bench: Contamination-proof, publicly verifiable, and privacy-preserving llm evaluation via zero-knowledge proofs, 2026. URL https://openreview.net/pdf?id=SAY3zZt2my. Primary PDF; protocol relation, Section 3.3.

Zkonduit Inc. EZKL: an engine for doing inference for deep learning models and other computational graphs in a zk-SNARK. https://github.com/zkonduit/ezkl, 2022. Software. Repository created July 2022; earliest release still listed on GitHub/PyPI is v7.1.x (January 2024).